

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Sergey Ioffe
Google Inc., sioffe@google.com

Christian Szegedy
Google Inc., szegedy@google.com

Abstract

Training Deep Neural Networks is complicated by the fact that the distribution of each layer’s inputs changes during training, as the parameters of the previous layers change. This slows down the training by requiring lower learning rates and careful parameter initialization, and makes it notoriously hard to train models with saturating nonlinearities. We refer to this phenomenon as *internal covariate shift*, and address the problem by normalizing layer inputs. Our method draws its strength from making normalization a part of the model architecture and performing the normalization *for each training mini-batch*. Batch Normalization allows us to use much higher learning rates and be less careful about initialization. It also acts as a regularizer, in some cases eliminating the need for Dropout. Applied to a state-of-the-art image classification model, Batch Normalization achieves the same accuracy with 14 times fewer training steps, and beats the original model by a significant margin. Using an ensemble of batch-normalized networks, we improve upon the best published result on ImageNet classification: reaching 4.9% top-5 validation error (and 4.8% test error), exceeding the accuracy of human raters.

1 Introduction

Deep learning has dramatically advanced the state of the art in vision, speech, and many other areas. Stochastic gradient descent (SGD) has proved to be an effective way of training deep networks, and SGD variants such as momentum (Sutskever et al., 2013) and Adagrad (Duchi et al., 2011) have been used to achieve state of the art performance. SGD optimizes the parameters Θ of the network, so as to minimize the loss

$$\Theta = \arg \min_{\Theta} \frac{1}{N} \sum_{i=1}^N \ell(x_i, \Theta)$$

where $x_1 \dots x_N$ is the training data set. With SGD, the training proceeds in steps, and at each step we consider a *mini-batch* $x_1 \dots x_m$ of size m . The mini-batch is used to approximate the gradient of the loss function with respect to the parameters, by computing

$$\frac{1}{m} \frac{\partial \ell(x_i, \Theta)}{\partial \Theta}.$$

Using mini-batches of examples, as opposed to one example at a time, is helpful in several ways. First, the gradient of the loss over a mini-batch is an estimate of the gradient over the training set, whose quality improves as the batch size increases. Second, computation over a batch can be much more efficient than m computations for individual examples, due to the parallelism afforded by the modern computing platforms.

While stochastic gradient is simple and effective, it requires careful tuning of the model hyper-parameters, specifically the learning rate used in optimization, as well as the initial values for the model parameters. The training is complicated by the fact that the inputs to each layer are affected by the parameters of all preceding layers – so that small changes to the network parameters amplify as the network becomes deeper.

The change in the distributions of layers’ inputs presents a problem because the layers need to continuously adapt to the new distribution. When the input distribution to a learning system changes, it is said to experience *covariate shift* (Shimodaira, 2000). This is typically handled via domain adaptation (Jiang, 2008). However, the notion of covariate shift can be extended beyond the learning system as a whole, to apply to its parts, such as a sub-network or a layer. Consider a network computing

$$\ell = F_2(F_1(u, \Theta_1), \Theta_2)$$

where F_1 and F_2 are arbitrary transformations, and the parameters Θ_1, Θ_2 are to be learned so as to minimize the loss ℓ . Learning Θ_2 can be viewed as if the inputs $x = F_1(u, \Theta_1)$ are fed into the sub-network

$$\ell = F_2(x, \Theta_2).$$

For example, a gradient descent step

$$\Theta_2 \leftarrow \Theta_2 - \frac{\alpha}{m} \sum_{i=1}^m \frac{\partial F_2(x_i, \Theta_2)}{\partial \Theta_2}$$

(for batch size m and learning rate α) is exactly equivalent to that for a stand-alone network F_2 with input x . Therefore, the input distribution properties that make training more efficient – such as having the same distribution between the training and test data – apply to training the sub-network as well. As such it is advantageous for the distribution of x to remain fixed over time. Then, Θ_2 does

批量归一化：通过降低内部协变量偏移来加速深度网络训练

Sergey Ioffe, Google
Inc., sioffe@google.com

Christian Szegedy, Google
Inc., szegedy@google.com

摘要

由于在训练过程中前几层参数的变动会导致各层输入数据的分布发生变化，深度神经网络的训练变得十分复杂。这种情况不仅要求使用更低的学习率并进行更为精细的参数初始化，从而延缓训练进度，还会给那些包含饱和非线性函数的模型带来极大的训练难度。我们将这种现象称为内部协变量偏移，并通过对接入层的输入数据进行归一化处理来解决该问题。我们的方法的优势在于将归一化处理整合到模型架构中，并对每个训练小批量都执行归一化操作。批量归一化使得我们可以采用更高的学习率，同时也无需那么注重参数初始化的问题。此外，它还具备正则化作用，在某些情况下甚至可以省去使用Dropout机制的必要。将该技术应用于当前最佳性能的图像分类模型后，批量归一化能够在训练步数仅为原来的1/14的情况下依然保持相同的准确率，且表现远超原始模型。再通过构建由经过批量归一化处理的模型组成的集成模型，我们进一步提升了在ImageNet图像分类任务上的最佳成果：将Top-5验证错误率降至4.9%，测试错误率降至4.8%，这一准确率甚至超过了人工评估员的评分水平。

1 引言

深度学习极大地推动了视觉、语音以及众多其他领域的先进技术发展。随机梯度下降法（SGD）已被证明是训练深度模型的有效方法，而诸如动量法（Sutskever等人，2013年）和Adagrad算法（Duchi等人，2011年）这类SGD的变体也被用来获得最优的性能表现。SGD通过优化网络中的 Θ 参数来最小化损失值。

$$\Theta = \arg \min_{\Theta} \frac{1}{N} \sum_{i=1}^N \ell(x_i, \Theta)$$

其中{xv1}指的是训练数据集。采用随机梯度下降时，训练是分步骤进行的，在每一步中我们会考虑一个大小为{xv3}的小批量{xv2}。通过计算，这个小批量可用于近似求出损失函数相对于各参数的梯度。

$$\frac{1}{m} \frac{\partial \ell(x_i, \Theta)}{\partial \Theta}.$$

与一次处理一个样本不同，使用包含多个样本的小批量在多个方面都具有优势。首先，基于小批量的损失函数梯度实际上是整个训练数据集上梯度的一个估计值，而随着批次大小的增加，这一估计值的精度也会随之提升。其次，得益于现代计算平台所提供的并行处理能力，对整个批次进行计算通常比逐个处理样本的{xv1}计算方式效率更高。

尽管随机梯度下降法简单且高效，但它在应用时需要仔细调整模型超参数，尤其是用于优化的学习率以及模型参数的初始值。更为复杂的是，由于每一层的输入都会受到所有前置层参数的影响，因此当网络参数发生微小变化时，这种变化会在网络结构越深的情况下被不断放大，进而增加训练难度。

各层输入分布的变化会带来一个问题，因为这些层必须持续适应新的分布情况。当学习系统的输入分布发生变化时，就称之为出现了协变量偏移现象（Shimodaira, 2000年）。这种情况通常通过领域自适应技术来解决（Jiang, 2008年）。不过，协变量偏移的概念不仅可以应用于整个学习系统，还可以延伸到其组成部分，比如子网络或单个层。试想在一个正在运行的网络中……

$$\ell = F_2(F_1(u, \Theta_1), \Theta_2)$$

其中 F_1 和 F_2 为任意变换，而参数 Θ_1 、 Θ_2 则需要被学习出来，以此最小化损失 ℓ 。可以将参数的学习 Θ_2 视为将输入 $x = F_1(u, \Theta_1)$ 输入到子网络中的过程。

$$\ell = F_2(x, \Theta_2).$$

例如，一个梯度下降步骤。

$$\Theta_2 \leftarrow \Theta_2 - \frac{\alpha}{m} \sum_{i=1}^m \frac{\partial F_2(x_i, \Theta_2)}{\partial \Theta_2}$$

（当批量大小为 m 且学习率为 α 时），其效果与针对输入为 x 的独立网络 F_2 的情况完全相同。因此，那些能够提升训练效率的输入 x 的分布特性——比如训练数据与测试数据的分布保持一致——同样适用于子网络的训练。正因如此，让 x 的分布随时间保持不变具有明显优势。这样一来， Θ_2 就无需再进行调整，以弥补 x 的分布变化所带来的影响。

not have to readjust to compensate for the change in the distribution of x .

Fixed distribution of inputs to a sub-network would have positive consequences for the layers *outside* the sub-network, as well. Consider a layer with a sigmoid activation function $z = g(Wu + b)$ where u is the layer input, the weight matrix W and bias vector b are the layer parameters to be learned, and $g(x) = \frac{1}{1+\exp(-x)}$. As $|x|$ increases, $g'(x)$ tends to zero. This means that for all dimensions of $x = Wu + b$ except those with small absolute values, the gradient flowing down to u will vanish and the model will train slowly. However, since x is affected by W , b and the parameters of all the layers below, changes to those parameters during training will likely move many dimensions of x into the saturated regime of the nonlinearity and slow down the convergence. This effect is amplified as the network depth increases. In practice, the saturation problem and the resulting vanishing gradients are usually addressed by using Rectified Linear Units (Nair & Hinton, 2010) $ReLU(x) = \max(x, 0)$, careful initialization (Bengio & Glorot, 2010; Saxe et al., 2013), and small learning rates. If, however, we could ensure that the distribution of nonlinearity inputs remains more stable as the network trains, then the optimizer would be less likely to get stuck in the saturated regime, and the training would accelerate.

We refer to the change in the distributions of internal nodes of a deep network, in the course of training, as *Internal Covariate Shift*. Eliminating it offers a promise of faster training. We propose a new mechanism, which we call *Batch Normalization*, that takes a step towards reducing internal covariate shift, and in doing so dramatically accelerates the training of deep neural nets. It accomplishes this via a normalization step that fixes the means and variances of layer inputs. Batch Normalization also has a beneficial effect on the gradient flow through the network, by reducing the dependence of gradients on the scale of the parameters or of their initial values. This allows us to use much higher learning rates without the risk of divergence. Furthermore, batch normalization regularizes the model and reduces the need for Dropout (Srivastava et al., 2014). Finally, Batch Normalization makes it possible to use saturating nonlinearities by preventing the network from getting stuck in the saturated modes.

In Sec. 4.2, we apply Batch Normalization to the best-performing ImageNet classification network, and show that we can match its performance using only 7% of the training steps, and can further exceed its accuracy by a substantial margin. Using an ensemble of such networks trained with Batch Normalization, we achieve the top-5 error rate that improves upon the best known results on ImageNet classification.

2 Towards Reducing Internal Covariate Shift

We define *Internal Covariate Shift* as the change in the distribution of network activations due to the change in network parameters during training. To improve the training, we seek to reduce the internal covariate shift. By fixing the distribution of the layer inputs x as the training progresses, we expect to improve the training speed. It has been long known (LeCun et al., 1998b; Wiesler & Ney, 2011) that the network training converges faster if its inputs are whitened – i.e., linearly transformed to have zero means and unit variances, and decorrelated. As each layer observes the inputs produced by the layers below, it would be advantageous to achieve the same whitening of the inputs of each layer. By whitening the inputs to each layer, we would take a step towards achieving the fixed distributions of inputs that would remove the ill effects of the internal covariate shift.

We could consider whitening activations at every training step or at some interval, either by modifying the network directly or by changing the parameters of the optimization algorithm to depend on the network activation values (Wiesler et al., 2014; Raiko et al., 2012; Povey et al., 2014; Desjardins & Kavukcuoglu). However, if these modifications are interspersed with the optimization steps, then the gradient descent step may attempt to update the parameters in a way that requires the normalization to be updated, which reduces the effect of the gradient step. For example, consider a layer with the input u that adds the learned bias b , and normalizes the result by subtracting the mean of the activation computed over the training data: $\hat{x} = x - E[x]$ where $x = u + b$, $\mathcal{X} = \{x_{1...N}\}$ is the set of values of x over the training set, and $E[x] = \frac{1}{N} \sum_{i=1}^N x_i$. If a gradient descent step ignores the dependence of $E[x]$ on b , then it will update $b \leftarrow b + \Delta b$, where $\Delta b \propto -\partial \ell / \partial \hat{x}$. Then $u + (b + \Delta b) - E[u + (b + \Delta b)] = u + b - E[u + b]$. Thus, the combination of the update to b and subsequent change in normalization led to no change in the output of the layer nor, consequently, the loss. As the training continues, b will grow indefinitely while the loss remains fixed. This problem can get worse if the normalization not only centers but also scales the activations. We have observed this empirically in initial experiments, where the model blows up when the normalization parameters are computed outside the gradient descent step.

The issue with the above approach is that the gradient descent optimization does not take into account the fact that the normalization takes place. To address this issue, we would like to ensure that, for any parameter values, the network *always* produces activations with the desired distribution. Doing so would allow the gradient of the loss with respect to the model parameters to account for the normalization, and for its dependence on the model parameters Θ . Let again x be a layer input, treated as a

无需重新调整来抵消 x 的分布变化。

如果子网络的输入分布保持固定，那么对该子网络之外的层也会产生积极影响。以一个采用Sigmoid函数作为激活函数的层为例，其表达式为 $z = g(Wu + b)$ ，其中 u 为该层的输入，权重矩阵 W 与偏差向量 b 则是需要被学习的层参数，而 $g(x) = \frac{1}{1+\exp(-x)}$ 则为对应的输出表达式。随着 $|x|$ 的值增大， $g'(x)$ 会趋向于零。这意味着，对于 $x = Wu + b$ 的所有维度而言，除了那些绝对值较小的维度外，流向 u 的梯度将会消失，从而导致模型训练速度变慢。然而，由于 x 会受到 W 、 b 以及所有下层节点参数的影响，训练过程中这些参数的变动很可能会使 x 的许多维度进入非线性函数的饱和区域，进而进一步降低收敛速度。随着网络深度的增加，这种效应还会愈发显著。在实际应用中，为了解决饱和问题以及由此引发的梯度消失问题，人们通常会采用修正线性单元（Nair & Hinton, 2010） $ReLU(x) = \max(x, 0)$ ，进行合理的参数初始化（Bengio & Glorot, 2010; Saxe等人, 2013），并设置较小的学习率。不过，如果我们能够确保在网络训练过程中非线性函数的输入分布始终保持稳定，那么优化器就不太可能陷入饱和状态，模型的训练速度也因而可以得到提升。

在深度网络训练过程中，内部节点分布所发生的变化被我们称为内部协变量偏移。消除这种偏移有望显著提升训练速度。我们提出了一种名为批量归一化的新机制，该机制通过有效降低内部协变量偏移，大幅加速了深度神经网络的训练进程。其实现方式是通过归一化处理来确定层输入的均值与方差。此外，批量归一化还能改善梯度在网络中的传播情况，因为它能减少梯度对参数规模或其初始值的依赖，从而使我们能够在不面临发散风险的前提下使用更高的学习率。同时，批量归一化还具有对模型进行正则化的作用，进而降低了使用Dropout机制的必要性（Srivastava等人, 2014年）。最后，由于能够防止网络陷入饱和状态，批量归一化使得使用饱和和非线性函数成为可能。

在4.2节中，我们将批量归一化技术应用于性能最佳的ImageNet图像分类网络，并证明仅需其7%的训练步数即可达到与其相当的性能，甚至还能在准确率上实现大幅超越。通过使用经过批量归一化训练的此类网络构成的集成模型，我们进一步将前5名错误率降低到了优于目前ImageNet图像分类任务中已知最佳结果的水平。

2 减少内部协变量偏移的路径

我们将内部协变量偏移定义为在训练过程中由于网络参数发生变化而导致的网络激活值分布发生改变的现象。为提升网络训练效果，我们需要努力减少这种内部协变量偏移。通过在整个训练过程中保持层输入 x 的分布不变，我们预计能够提高训练速度。长期以来人们已经知道（LeCun等人, 1998b; Wiesler与Ney, 2011），如果让网络输入经过白化处理——也就是进行线性变换使其均值变为零、方差为1且各数值之间不相关——那么网络训练将会更快收敛。由于每一层都需要接收下层节点产生的输入，因此让每一层的输入都实现同样的白化处理会非常有好处。通过对每层输入进行白化处理，我们就能朝实现固定输入分布的目标迈进，从而消除内部协变量偏移带来的不良影响。

我们可以考虑在每个训练步骤或特定间隔对激活值进行白化处理，方法包括直接修改网络结构，或是调整优化算法的参数使其依赖于网络的激活值（Wiesler等人, 2014年; Raiko等人, 2012年; Povey等人, 2014年; Desjardins与Kavukcuoglu）。然而，如果这些修改与优化步骤交错进行，那么在梯度下降步骤中尝试更新参数时可能会要求同时更新归一化参数，这样一来反而会削弱梯度步骤的作用。例如，设想一个输入为 u 的层，该层会先加入学习得到的偏差 b ，再通过减去基于训练数据计算出的激活值均值 $\hat{x} = x - E[x]$ 来对结果进行归一化：其中 $x = u + b$ ， $\mathcal{X} = \{x_{1...N}\}$ 表示训练集中 x 的所有取值， $E[x] = \frac{1}{N} \sum_{i=1}^N x_i$ 则表示对应的期望值。倘若在梯度下降步骤中忽略了 $E[x]$ 与 b 之间的关联，它就会更新 $b \leftarrow b + \Delta b$ ，此时 $u + (b + \Delta b) - E[u + (b + \Delta b)] = u + b - E[u + b]$ 。如此一来，对 b 的更新与随后的归一化变化相结合，既不会改变该层的输出，也不会影响损失函数的值。随着训练的持续进行， b 的值将会无限增大，而损失值却保持不变。如果归一化处理不仅用于将激活值居中，还用于对其进行缩放，这种情况还会更加严重。我们在初步实验中就通过实验证实了这一点：当在梯度下降步骤之外计算归一化参数时，模型就会出现数值爆炸现象。

上述方法存在的问题在于，梯度下降优化并未考虑到实际已经进行了归一化处理这一事实。为了解决这一问题，我们需要确保对于任何参数值，网络始终能够生成具有期望的分布的激活值。这样一来，损失函数关于模型参数的梯度就能同时体现归一化处理的影响，以及该处理对模型参数的依赖关系 Θ 。此处仍以 x 作为层输入，将其视作一个向量，而 Θ 则表示训练数据集中所有此类输入的集合。此时，归一化处理就可以被表达为一种变换形式。

vector, and $\mathcal{X}$ be the set of these inputs over the training data set. The normalization can then be written as a transformation

$$\hat{\mathbf{x}} = \text{Norm}(\mathbf{x}, \mathcal{X})$$

which depends not only on the given training example $\mathbf{x}$ but on all examples $\mathcal{X}$ – each of which depends on Θ if $\mathbf{x}$ is generated by another layer. For backpropagation, we would need to compute the Jacobians

$$\frac{\partial \text{Norm}(\mathbf{x}, \mathcal{X})}{\partial \mathbf{x}} \text{ and } \frac{\partial \text{Norm}(\mathbf{x}, \mathcal{X})}{\partial \mathcal{X}};$$

ignoring the latter term would lead to the explosion described above. Within this framework, whitening the layer inputs is expensive, as it requires computing the covariance matrix $\text{Cov}[\mathbf{x}] = \text{E}_{\mathbf{x} \in \mathcal{X}}[\mathbf{x}\mathbf{x}^T] - \text{E}[\mathbf{x}]\text{E}[\mathbf{x}]^T$ and its inverse square root, to produce the whitened activations $\text{Cov}[\mathbf{x}]^{-1/2}(\mathbf{x} - \text{E}[\mathbf{x}])$, as well as the derivatives of these transforms for backpropagation. This motivates us to seek an alternative that performs input normalization in a way that is differentiable and does not require the analysis of the entire training set after every parameter update.

Some of the previous approaches (e.g. (Lyu & Simoncelli, 2008)) use statistics computed over a single training example, or, in the case of image networks, over different feature maps at a given location. However, this changes the representation ability of a network by discarding the absolute scale of activations. We want to preserve the information in the network, by normalizing the activations in a training example relative to the statistics of the entire training data.

3 Normalization via Mini-Batch Statistics

Since the full whitening of each layer’s inputs is costly and not everywhere differentiable, we make two necessary simplifications. The first is that instead of whitening the features in layer inputs and outputs jointly, we will normalize each scalar feature independently, by making it have the mean of zero and the variance of 1. For a layer with d -dimensional input $\mathbf{x} = (x^{(1)} \dots x^{(d)})$, we will normalize each dimension

$$\hat{x}^{(k)} = \frac{x^{(k)} - \text{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

where the expectation and variance are computed over the training data set. As shown in (LeCun et al., 1998b), such normalization speeds up convergence, even when the features are not decorrelated.

Note that simply normalizing each input of a layer may change what the layer can represent. For instance, normalizing the inputs of a sigmoid would constrain them to the linear regime of the nonlinearity. To address this, we make sure that *the transformation inserted in the network can represent the identity transform*. To accomplish this,

we introduce, for each activation $x^{(k)}$, a pair of parameters $\gamma^{(k)}, \beta^{(k)}$, which scale and shift the normalized value:

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}.$$

These parameters are learned along with the original model parameters, and restore the representation power of the network. Indeed, by setting $\gamma^{(k)} = \sqrt{\text{Var}[x^{(k)}]}$ and $\beta^{(k)} = \text{E}[x^{(k)}]$, we could recover the original activations, if that were the optimal thing to do.

In the batch setting where each training step is based on the entire training set, we would use the whole set to normalize activations. However, this is impractical when using stochastic optimization. Therefore, we make the second simplification: since we use mini-batches in stochastic gradient training, *each mini-batch produces estimates of the mean and variance* of each activation. This way, the statistics used for normalization can fully participate in the gradient backpropagation. Note that the use of mini-batches is enabled by computation of per-dimension variances rather than joint covariances; in the joint case, regularization would be required since the mini-batch size is likely to be smaller than the number of activations being whitened, resulting in singular covariance matrices.

Consider a mini-batch $\mathcal{B}$ of size m . Since the normalization is applied to each activation independently, let us focus on a particular activation $x^{(k)}$ and omit k for clarity. We have m values of this activation in the mini-batch,

$$\mathcal{B} = \{x_{1\dots m}\}.$$

Let the normalized values be $\hat{x}_{1\dots m}$, and their linear transformations be $y_{1\dots m}$. We refer to the transform

$$\text{BN}_{\gamma, \beta} : x_{1\dots m} \rightarrow y_{1\dots m}$$

as the *Batch Normalizing Transform*. We present the BN Transform in Algorithm 1. In the algorithm, ϵ is a constant added to the mini-batch variance for numerical stability.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$; Parameters to be learned: γ, β Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$ $\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$ $\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$ $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$ $y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$
--

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

The BN transform can be added to a network to manipulate any activation. In the notation $y = \text{BN}_{\gamma, \beta}(x)$, we

归一化处理因此可以被表述为一种变换操作。

$$\hat{\mathbf{x}} = \text{Norm}(\mathbf{x}, \mathcal{X})$$

这一结果不仅取决于给定的训练样本 $\mathbf{x}$, 还与所有样本 $\mathcal{X}$ 有关——如果 $\mathbf{x}$ 是由其他层生成的, 那么每个样本又会依赖于 Θ 。为了进行反向传播, 我们必须计算这些雅可比矩阵。

$$\frac{\partial \text{Norm}(\mathbf{x}, \mathcal{X})}{\partial \mathbf{x}} \text{ and } \frac{\partial \text{Norm}(\mathbf{x}, \mathcal{X})}{\partial \mathcal{X}};$$

若忽略后一项, 就会引发上文所述的数值爆炸问题。在此框架下, 对层输入进行白化处理成本较高, 因为需要计算协方差矩阵 $\text{Cov}[\mathbf{x}] = \text{E}_{\mathbf{x} \in \mathcal{X}}[\mathbf{x}\mathbf{x}^T] - \text{E}[\mathbf{x}]\text{E}[\mathbf{x}]^T$ 及其平方根的倒数, 从而得到经过白化处理的激活值 $\text{Cov}[\mathbf{x}]^{-1/2}(\mathbf{x} - \text{E}[\mathbf{x}])$, 同时还需为反向传播计算这些变换的导数。正因如此, 我们寻求一种替代方案, 即以可微分的方式实现输入归一化处理, 且无需在每次参数更新后都对整个训练集进行分析。

此前的一些方法(例如Lyu和Simoncelli在2008年提出的方法)会利用单个训练样本的统计信息, 或者在图像网络中利用某一位置不同特征图的统计信息。然而, 这种做法会通过丢弃激活值的绝对尺度来改变网络的表示能力。我们希望通过训练样本中的激活值相对于整个训练数据的统计信息进行归一化, 从而保留网络中的信息。

3 小批量统计法实现的归一化处理

由于对每层的层输入进行完全白化处理成本高昂且并非处处都可微分, 因此我们需要做出两项必要的简化。首先, 我们不再同时对层输入和输出中的特征进行白化处理, 而是分别对每个标量特征进行独立归一化, 使其均值变为0、方差变为1。对于具有 d 维输入 $\mathbf{x} = (x^{(1)} \dots x^{(d)})$ 的层, 我们将对每一维度都进行归一化处理。

$$\hat{x}^{(k)} = \frac{x^{(k)} - \text{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

其中期望值和方差都是基于训练数据集计算得出的。正如 (LeCun等人, 1998b)所指出的, 即便这些特征之间并不相关, 这样的归一化处理也能提升模型的收敛速度。

需要注意的是, 仅对某一层的所有输入进行标准化处理, 就可能会改变该层所能表示的内容。例如, 对 Sigmoid函数的输入进行标准化会将其限制在非线性函数的线性区间内。为了解决这一问题, 我们会确保插入网络中的变换能够实现恒等变换, 从而达到上述目标。

针对每一种激活值 $x^{(k)}$, 我们会引入一组由参数 $\gamma^{(k)}$ 、 $\beta^{(k)}$ 构成的参数对, 用于对标准化后的数值进行缩放与平移操作。

$$y^{(k)} = \gamma^{(k)}\hat{x}^{(k)} + \beta^{(k)}.$$

这些参数会与原有的模型参数一同被学习, 从而恢复网络的表征能力。实际上, 只要设定 $\gamma^{(k)} = \sqrt{\text{Var}[x^{(k)}]}$ 和 $\beta^{(k)} = \text{E}[x^{(k)}]$, 我们就能够恢复原有的激活值, 当然前提是这是最理想的做法。

在每个训练步骤都基于整个训练集的批量处理模式下, 我们会使用整个数据集来对激活值进行归一化处理。不过, 在采用随机优化算法时, 这种方式并不实用。因此我们做出了第二个简化假设: 由于在基于随机梯度的训练中会使用小批量数据, 每个小批量都能分别计算出各激活值的均值和方差。这样一来, 用于归一化处理的统计量就能充分参与到梯度反向传播的过程中。需要注意的是, 之所以能够使用小批量数据, 是因为可以通过计算各维度的方差而非联合协方差来实现; 如果在处理联合协方差的情况下使用小批量, 由于小批量大小往往小于需要做归一化的激活值数量, 从而导致协方差矩阵出现奇异, 此时就需要引入正则化处理。

假设有一个大小为 m 的小批量 $\mathcal{B}$ 。由于归一化是针对每个激活值单独进行的, 为便于说明, 我们暂且只关注某个特定的激活值 $x^{(k)}$, 而忽略 k 。在这个小批量中, 该激活值共有 m 个样本值。

$$\mathcal{B} = \{x_{1\dots m}\}.$$

设这些经过归一化的值为 $\hat{x}_{1\dots m}$, 其线性变换为 $y_{1\dots m}$ 。我们将这种变换称为

$$\text{BN}_{\gamma, \beta} : x_{1\dots m} \rightarrow y_{1\dots m}$$

批量归一化变换。算法1中介绍了具体的批归一化变换方式, 在该算法中, ϵ 是一个为保证数值稳定性而加到小批量方差上的常数。

输入: 一个小批量内的 x 数值: $\mathcal{B} = \{x_{1\dots m}\}$; 需要学习的参数: γ, β 输出: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$ $\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ 小批量均值}$ $\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ 小批量的方差}$ $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ 进行归一化}$ $y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{批量归一化}_{\gamma, \beta}(x_i) \quad // \text{ 执行缩放与平移操作}$

算法1: 批量归一化变换, 该变换被应用于小批量数据中的激活值 x 。

批归一化变换可以被添加到神经网络中, 用以调整任何激活值。在记号 $y = \text{BN}_{\gamma, \beta}(x)$ 中, 其含义为

indicate that the parameters γ and β are to be learned, but it should be noted that the BN transform does not independently process the activation in each training example. Rather, $\text{BN}_{\gamma,\beta}(x)$ depends both on the training example *and the other examples in the mini-batch*. The scaled and shifted values y are passed to other network layers. The normalized activations $\hat{x}$ are internal to our transformation, but their presence is crucial. The distributions of values of any $\hat{x}$ has the expected value of 0 and the variance of 1, as long as the elements of each mini-batch are sampled from the same distribution, and if we neglect ϵ . This can be seen by observing that $\sum_{i=1}^m \hat{x}_i = 0$ and $\frac{1}{m} \sum_{i=1}^m \hat{x}_i^2 = 1$, and taking expectations. Each normalized activation $\hat{x}^{(k)}$ can be viewed as an input to a sub-network composed of the linear transform $y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$, followed by the other processing done by the original network. These sub-network inputs all have fixed means and variances, and although the joint distribution of these normalized $\hat{x}^{(k)}$ can change over the course of training, we expect that the introduction of normalized inputs accelerates the training of the sub-network and, consequently, the network as a whole.

During training we need to backpropagate the gradient of loss ℓ through this transformation, as well as compute the gradients with respect to the parameters of the BN transform. We use chain rule, as follows (before simplification):

$$\begin{aligned}\frac{\partial \ell}{\partial \hat{x}_i} &= \frac{\partial \ell}{\partial y_i} \cdot \gamma \\ \frac{\partial \ell}{\partial \sigma_B^2} &= \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_B) \cdot \frac{-1}{2} (\sigma_B^2 + \epsilon)^{-3/2} \\ \frac{\partial \ell}{\partial \mu_B} &= \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_B)}{m} \\ \frac{\partial \ell}{\partial x_i} &= \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial \ell}{\partial \mu_B} \cdot \frac{1}{m} \\ \frac{\partial \ell}{\partial \gamma} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i \\ \frac{\partial \ell}{\partial \beta} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}\end{aligned}$$

Thus, BN transform is a differentiable transformation that introduces normalized activations into the network. This ensures that as the model is training, layers can continue learning on input distributions that exhibit less internal covariate shift, thus accelerating the training. Furthermore, the learned affine transform applied to these normalized activations allows the BN transform to represent the identity transformation and preserves the network capacity.

3.1 Training and Inference with Batch-Normalized Networks

To *Batch-Normalize* a network, we specify a subset of activations and insert the BN transform for each of them, according to Alg. 1. Any layer that previously received x as the input, now receives $\text{BN}(x)$. A model employing Batch Normalization can be trained using batch gradient descent, or Stochastic Gradient Descent with a mini-batch size $m > 1$, or with any of its variants such as Adagrad

(Duchi et al., 2011). The normalization of activations that depends on the mini-batch allows efficient training, but is neither necessary nor desirable during inference; we want the output to depend only on the input, deterministically. For this, once the network has been trained, we use the normalization

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}$$

using the population, rather than mini-batch, statistics. Neglecting ϵ , these normalized activations have the same mean 0 and variance 1 as during training. We use the unbiased variance estimate $\text{Var}[x] = \frac{m}{m-1} \cdot \mathbb{E}_B[\sigma_B^2]$, where the expectation is over training mini-batches of size m and σ_B^2 are their sample variances. Using moving averages instead, we can track the accuracy of a model as it trains. Since the means and variances are fixed during inference, the normalization is simply a linear transform applied to each activation. It may further be composed with the scaling by γ and shift by β , to yield a single linear transform that replaces $\text{BN}(x)$. Algorithm 2 summarizes the procedure for training batch-normalized networks.

Input: Network N with trainable parameters Θ ; subset of activations $\{x^{(k)}\}_{k=1}^K$
Output: Batch-normalized network for inference, $N_{\text{BN}}^{\text{inf}}$

```

1:  $N_{\text{BN}}^{\text{tr}} \leftarrow N$  // Training BN network
2: for  $k = 1 \dots K$  do
3:   Add transformation  $y^{(k)} = \text{BN}_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})$  to  $N_{\text{BN}}^{\text{tr}}$  (Alg. 1)
4:   Modify each layer in  $N_{\text{BN}}^{\text{tr}}$  with input  $x^{(k)}$  to take  $y^{(k)}$  instead
5: end for
6: Train  $N_{\text{BN}}^{\text{tr}}$  to optimize the parameters  $\Theta \cup \{\gamma^{(k)}, \beta^{(k)}\}_{k=1}^K$ 
7:  $N_{\text{BN}}^{\text{inf}} \leftarrow N_{\text{BN}}^{\text{tr}}$  // Inference BN network with frozen // parameters

8: for  $k = 1 \dots K$  do
9:   // For clarity,  $x \equiv x^{(k)}, \gamma \equiv \gamma^{(k)}, \mu_B \equiv \mu_B^{(k)}$ , etc.
10:  Process multiple training mini-batches  $\mathcal{B}$ , each of size  $m$ , and average over them:
        
$$\begin{aligned}\mathbb{E}[x] &\leftarrow \mathbb{E}_{\mathcal{B}}[\mu_{\mathcal{B}}] \\ \text{Var}[x] &\leftarrow \frac{m}{m-1} \mathbb{E}_{\mathcal{B}}[\sigma_{\mathcal{B}}^2]\end{aligned}$$

11:  In  $N_{\text{BN}}^{\text{inf}}$ , replace the transform  $y = \text{BN}_{\gamma, \beta}(x)$  with
        
$$y = \frac{\gamma}{\sqrt{\text{Var}[x] + \epsilon}} \cdot x + \left( \beta - \frac{\gamma \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \right)$$

12: end for
```

Algorithm 2: Training a Batch-Normalized Network

3.2 Batch-Normalized Convolutional Networks

Batch Normalization can be applied to any set of activations in the network. Here, we focus on transforms

这表明参数 γ 和 β 需要被学习，但需要注意的是，批归一化变换并不会单独处理每个训练样本中的激活值。实际上， $\text{BN}_{\gamma,\beta}(x)$ 的计算既依赖于当前训练样本，也依赖于小批量中的其他样本。经过缩放和平移后的值 y 会被传递给其他网络层。归一化后的激活值 $\hat{x}$ 虽然属于该变换的内部输出，但其存在至关重要。只要每个小批量中的元素都来自相同的分布且忽略 ϵ ，那么任何 $\hat{x}$ 的数值分布都具有期望值为0、方差为1的特性。这一点可以通过观察 $\sum_{i=1}^m \hat{x}_i = 0$ 和 $\frac{1}{m} \sum_{i=1}^m \hat{x}_i^2 = 1$ 并取期望值来验证。每一个归一化后的激活值 $\hat{x}^{(k)}$ 都可以被视为一个子网络的输入，该子网络首先经过线性变换 $y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$ ，随后再执行原始网络的其他处理步骤。这些子网络的输入均具有固定的均值和方差，尽管这些归一化后的 $\hat{x}^{(k)}$ 的联合分布会在训练过程中发生变化，但我们预计引入归一化后的输入能够加速子网络的训练，进而提升整个网络的训练效率。

在训练过程中，我们需要将损失函数的梯度 ℓ 通过这种变换反向传播，同时计算出与批归一化变换的参数相关的梯度值。为此我们会运用链式法则，具体过程如下（简化后）：

$$\begin{aligned}\frac{\partial \ell}{\partial \hat{x}_i} &= \frac{\partial \ell}{\partial y_i} \cdot \gamma \\ \frac{\partial \ell}{\partial \sigma_B^2} &= \sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot (x_i - \mu_B) \cdot \frac{-1}{2} (\sigma_B^2 + \epsilon)^{-3/2} \\ \frac{\partial \ell}{\partial \mu_B} &= \left(\sum_{i=1}^m \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{-1}{\sqrt{\sigma_B^2 + \epsilon}} \right) + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{\sum_{i=1}^m -2(x_i - \mu_B)}{m} \\ \frac{\partial \ell}{\partial x_i} &= \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial \ell}{\partial \mu_B} \cdot \frac{1}{m} \\ \frac{\partial \ell}{\partial \gamma} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i} \cdot \hat{x}_i \\ \frac{\partial \ell}{\partial \beta} &= \sum_{i=1}^m \frac{\partial \ell}{\partial y_i}\end{aligned}$$

因此，批归一化变换是一种可微分的变换，它能够为网络引入归一化激活值。这样一来，在模型训练期间，各层即便面对存在较小内部协变量偏移的输入数据，也能继续进行学习，从而加快训练速度。此外，施加在这些归一化激活值上的学习得到的仿射变换，使得批归一化变换能够实现恒等变换的功能，并保留网络原有的表达能力。

3.1 使用批归一化网络进行训练与推理

要对模型进行批量归一化处理，需依据算法1的规定，选定部分激活值并在其对应位置插入批归一化变换。此前以 x 作为输入的任意层，现在则会接收 $\text{BN}(x)$ 作为输入。采用批量归一化的模型可以通过批量梯度下降法进行训练，也可使用小批量大小为 $m > 1$ 的随机梯度下降法，或是Adagrad算法这类其他变体来进行训练。

(Duchi等人，2011年)。那种依赖于小批量的激活值归一化处理虽能提升训练效率，但在推理阶段既非必需也不理想；因为我们希望输出结果能够仅由输入决定，且具有确定性。为此，在模型训练完成后，我们会使用不依赖小批量的归一化处理方式。

$$\hat{x} = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}}$$

此处采用总体统计量而非小批量统计量。忽略 ϵ 的影响后，这些归一化激活值的均值仍为0，方差也为1，与训练时的数值相同。我们使用无偏方差估计公式 $\text{Var}[x] = \frac{m}{m-1} \cdot \mathbb{E}_B[\sigma_B^2]$ ，其中期望值是对大小为 m 的训练小批量取的，而 σ_B^2 则是对样本的方差值。如果采用移动平均值，就可以在模型训练过程中实时监控其准确率。由于在推理阶段均值和方差是固定的，因此归一化处理实际上只是对每个激活值施加一次线性变换。该变换还可以进一步结合 γ 进行的缩放操作以及 β 进行的平移操作，最终得到一个可以替代 $\text{BN}(x)$ 的单一线性变换。算法2总结了训练批归一化网络的完整流程。

输入：具有可训练参数的网络 N Θ 以及部分激活值 $\{x^{(k)}\}_{k=1}^K$ ；输出：用于推理的批归一化网络 $N_{\text{BN}}^{\text{inf}}$

```

1:  $N_{\text{BN}}^{\text{tr}} \leftarrow N$  // 训练用批归一化网络
2: for  $k = 1 \dots K$  do
3:  将变换  $y^{(k)} = \text{BN}_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})$  添加到网络  $N_{\text{BN}}^{\text{tr}}$  中 (算法1)
4:  使用输入  $x^{(k)}$  修改网络  $N_{\text{BN}}^{\text{tr}}$  中的每一层，使其取  $y^{(k)}$  作为输出
5: end for
6: 训练网络  $N_{\text{BN}}^{\text{tr}}$  以优化参数  $\Theta \cup \{\gamma^{(k)}, \beta^{(k)}\}_{k=1}^K$ 
7:  $N_{\text{BN}}^{\text{inf}} \leftarrow N_{\text{BN}}^{\text{tr}}$  // 参数已被冻结的推理用批归一化网络
8: for  $k = 1 \dots K$  do
9:  // 为便于理解，此处指  $x \equiv x^{(k)}, \gamma \equiv \gamma^{(k)}, \mu_B \equiv \mu_B^{(k)}$  等
10: 处理多个大小为  $m$  的训练小批量  $\mathcal{B}$ ，并对这些数据求平均值：
        
$$\begin{aligned}\mathbb{E}[x] &\leftarrow \mathbb{E}_{\mathcal{B}}[\mu_{\mathcal{B}}] \\ \text{Var}[x] &\leftarrow \frac{m}{m-1} \mathbb{E}_{\mathcal{B}}[\sigma_{\mathcal{B}}^2]\end{aligned}$$

11: 在网络  $N_{\text{BN}}^{\text{inf}}$  中，用  $y = \frac{\gamma}{\sqrt{\text{Var}[x] + \epsilon}} \cdot x + \left( \beta - \frac{\gamma \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \right)$  替换原有的变换  $y = \text{BN}_{\beta}(x)$ 
12: end for
```

算法2：训练批归一化网络

3.2 批归一化卷积网络

批量归一化可以应用于网络中的任意一组激活值。在此，我们重点研究由仿射变换后接逐元素非线性运算所构成的变换。

that consist of an affine transformation followed by an element-wise nonlinearity:

$$\mathbf{z} = g(W\mathbf{u} + \mathbf{b})$$

where W and $\mathbf{b}$ are learned parameters of the model, and $g(\cdot)$ is the nonlinearity such as sigmoid or ReLU. This formulation covers both fully-connected and convolutional layers. We add the BN transform immediately before the nonlinearity, by normalizing $\mathbf{x} = W\mathbf{u} + \mathbf{b}$. We could have also normalized the layer inputs $\mathbf{u}$, but since $\mathbf{u}$ is likely the output of another nonlinearity, the shape of its distribution is likely to change during training, and constraining its first and second moments would not eliminate the covariate shift. In contrast, $W\mathbf{u} + \mathbf{b}$ is more likely to have a symmetric, non-sparse distribution, that is “more Gaussian” (Hyvärinen & Oja, 2000); normalizing it is likely to produce activations with a stable distribution.

Note that, since we normalize $W\mathbf{u} + \mathbf{b}$, the bias $\mathbf{b}$ can be ignored since its effect will be canceled by the subsequent mean subtraction (the role of the bias is subsumed by β in Alg. 1). Thus, $\mathbf{z} = g(W\mathbf{u} + \mathbf{b})$ is replaced with

$$\mathbf{z} = g(\text{BN}(W\mathbf{u}))$$

where the BN transform is applied independently to each dimension of $\mathbf{x} = W\mathbf{u}$, with a separate pair of learned parameters $\gamma^{(k)}, \beta^{(k)}$ per dimension.

For convolutional layers, we additionally want the normalization to obey the convolutional property – so that different elements of the same feature map, at different locations, are normalized in the same way. To achieve this, we jointly normalize all the activations in a mini-batch, over all locations. In Alg. 1, we let $\mathcal{B}$ be the set of all values in a feature map across both the elements of a mini-batch and spatial locations – so for a mini-batch of size m and feature maps of size $p \times q$, we use the effective mini-batch of size $m' = |\mathcal{B}| = m \cdot p \cdot q$. We learn a pair of parameters $\gamma^{(k)}, \beta^{(k)}$ per feature map, rather than per activation. Alg. 2 is modified similarly, so that during inference the BN transform applies the same linear transformation to each activation in a given feature map.

3.3 Batch Normalization enables higher learning rates

In traditional deep networks, too-high learning rate may result in the gradients that explode or vanish, as well as getting stuck in poor local minima. Batch Normalization helps address these issues. By normalizing activations throughout the network, it prevents small changes to the parameters from amplifying into larger and suboptimal changes in activations in gradients; for instance, it prevents the training from getting stuck in the saturated regimes of nonlinearities.

Batch Normalization also makes training more resilient to the parameter scale. Normally, large learning rates may increase the scale of layer parameters, which then amplify

the gradient during backpropagation and lead to the model explosion. However, with Batch Normalization, backpropagation through a layer is unaffected by the scale of its parameters. Indeed, for a scalar a ,

$$\text{BN}(W\mathbf{u}) = \text{BN}((aW)\mathbf{u})$$

and we can show that

$$\begin{aligned} \frac{\partial \text{BN}((aW)\mathbf{u})}{\partial \mathbf{u}} &= \frac{\partial \text{BN}(W\mathbf{u})}{\partial \mathbf{u}} \\ \frac{\partial \text{BN}((aW)\mathbf{u})}{\partial (aW)} &= \frac{1}{a} \cdot \frac{\partial \text{BN}(W\mathbf{u})}{\partial W} \end{aligned}$$

The scale does not affect the layer Jacobian nor, consequently, the gradient propagation. Moreover, larger weights lead to *smaller* gradients, and Batch Normalization will stabilize the parameter growth.

We further conjecture that Batch Normalization may lead the layer Jacobians to have singular values close to 1, which is known to be beneficial for training (Saxe et al., 2013). Consider two consecutive layers with normalized inputs, and the transformation between these normalized vectors: $\hat{\mathbf{z}} = F(\hat{\mathbf{x}})$. If we assume that $\hat{\mathbf{x}}$ and $\hat{\mathbf{z}}$ are Gaussian and uncorrelated, and that $F(\hat{\mathbf{x}}) \approx J\hat{\mathbf{x}}$ is a linear transformation for the given model parameters, then both $\hat{\mathbf{x}}$ and $\hat{\mathbf{z}}$ have unit covariances, and $I = \text{Cov}[\hat{\mathbf{z}}] = J\text{Cov}[\hat{\mathbf{x}}]J^T = JJ^T$. Thus, $JJ^T = I$, and so all singular values of J are equal to 1, which preserves the gradient magnitudes during backpropagation. In reality, the transformation is not linear, and the normalized values are not guaranteed to be Gaussian nor independent, but we nevertheless expect Batch Normalization to help make gradient propagation better behaved. The precise effect of Batch Normalization on gradient propagation remains an area of further study.

3.4 Batch Normalization regularizes the model

When training with Batch Normalization, a training example is seen in conjunction with other examples in the mini-batch, and the training network no longer producing deterministic values for a given training example. In our experiments, we found this effect to be advantageous to the generalization of the network. Whereas Dropout (Srivastava et al., 2014) is typically used to reduce overfitting, in a batch-normalized network we found that it can be either removed or reduced in strength.

4 Experiments

4.1 Activations over time

To verify the effects of internal covariate shift on training, and the ability of Batch Normalization to combat it, we considered the problem of predicting the digit class on the MNIST dataset (LeCun et al., 1998a). We used a very simple network, with a 28x28 binary image as input, and

即先进行仿射变换，再执行逐元素非线性运算的这种变换结构。

$$\mathbf{z} = g(W\mathbf{u} + \mathbf{b})$$

其中 W 和 $\mathbf{b}$ 是模型中需要学习的参数，而 $g(\cdot)$ 则表示如 Sigmoid 函数或 ReLU 函数之类的非线性函数。该公式同时适用于全连接层和卷积层。我们会在非线性函数之前立即进行批归一化变换，通过对 $\mathbf{x} = W\mathbf{u} + \mathbf{b}$ 进行归一化来实现这一操作。虽然我们本也可以对层输入 $\mathbf{u}$ 进行归一化，但由于 $\mathbf{u}$ 很可能是另一个非线性函数的输出，其在训练过程中的分布形态很可能会发生变化，且仅约束其一阶矩和二阶矩并不能消除协变量偏移问题。相比之下， $W\mathbf{u} + \mathbf{b}$ 更可能呈现出对称且非稀疏的分布特征，也就是更具高斯特性（Hyvärinen & Oja, 2000）；对其归一化处理后，所得到的激活值往往具有更为稳定的分布。

需要注意的是，由于我们对 $W\mathbf{u} + \mathbf{b}$ 进行了归一化处理，因此可以忽略偏差 $\mathbf{b}$ 的影响，因为其效应会被后续的均值减法所抵消（在算法1中，偏差的作用已被 β 承担）。这样一来， $\mathbf{z} = g(W\mathbf{u} + \mathbf{b})$ 就可以被替换为。

$$\mathbf{z} = g(\text{BN}(W\mathbf{u}))$$

在此过程中，批量归一化变换会独立地作用于 $\{\mathbf{xv1}\}$ 的每一个维度，且每个维度都对应一对独立的待学习参数 $\{\mathbf{xv2}\}, \{\mathbf{xv3}\}$ 。

对于卷积层而言，我们还希望归一化操作遵循卷积特性——即同一特征图中不同位置上的元素能够以相同的方式被归一化。为实现这一目标，我们会对小批量中所有位置的所有激活值进行统一归一化。在算法1中， $\mathcal{B}$ 指的是小批量中的各个元素以及不同空间位置上所有特征图中的数值集合；因此，当小批量大小为 $\{\mathbf{xv2}\}$ 且特征图大小为 $\{\mathbf{xv3}\}$ 时，实际使用的有效小批量大小则为 $\{\mathbf{xv4}\}$ 。我们为每个特征图学习一对参数 $\{\mathbf{xv5}\}$ 和 $\{\mathbf{xv6}\}$ ，而非仅为每个激活值单独学习参数。算法2也经过了类似修改，这样在推理阶段，批量归一化变换就能对给定特征图中的每个激活值应用相同的线性变换。

3.3 批量归一化使得学习率能够更高

在传统的深度网络中，过高的学习率可能导致梯度出现爆炸或消失的情况，同时还会使模型陷入较差的局部最小值。批量归一化有助于解决这些问题。通过对网络中的所有激活值进行归一化处理，它能够防止参数的微小变化被放大，进而导致激活值和梯度中出现更大且次优的变化；例如，它能够避免训练过程陷入非线性函数的饱和和区间。

批量归一化处理还能提升模型对参数尺度变化的抵御能力。通常情况下，过高的学习率会增大各层参数的尺度，进而加剧反向传播过程中的梯度数值爆炸现象。

而借助批量归一化，反向传播经过某一层时不会受到该层参数尺度的影响。实际上，对于一个标量 a 而言，

$$\text{BN}(W\mathbf{u}) = \text{BN}((aW)\mathbf{u})$$

而且我们能够证明这一点。

$$\begin{aligned} \frac{\partial \text{BN}((aW)\mathbf{u})}{\partial \mathbf{u}} &= \frac{\partial \text{BN}(W\mathbf{u})}{\partial \mathbf{u}} \\ \frac{\partial \text{BN}((aW)\mathbf{u})}{\partial (aW)} &= \frac{1}{a} \cdot \frac{\partial \text{BN}(W\mathbf{u})}{\partial W} \end{aligned}$$

归一化处理不会影响层雅可比矩阵，因此也不会影响梯度传播。此外，较大的权重会导致较小的梯度值，而批量归一化则能够稳定参数的增长速度。

我们进一步推测，批量归一化可能会让各层的雅可比矩阵的奇异值接近1，而这一特性已被证实对训练有益（Saxe等人，2013年）。以两个输入已经过归一化的连续层为例，这两个归一化向量之间的变换关系为 $\hat{\mathbf{z}} = F(\hat{\mathbf{x}})$ 。如果我们假设 $\hat{\mathbf{x}}$ 与 $\hat{\mathbf{z}}$ 是高斯分布且不相关的，并且 $F(\hat{\mathbf{x}}) \approx J\hat{\mathbf{x}}$ 是对应模型参数下的线性变换，那么 $\hat{\mathbf{x}}$ 与 $\hat{\mathbf{z}}$ 的协方差均为1，此时有 $I = \text{Cov}[\hat{\mathbf{z}}] = J\text{Cov}[\hat{\mathbf{x}}]J^T = JJ^T$ 。因此 $JJ^T = I$ ，也就是说 J 的所有奇异值都等于1，这能够在反向传播过程中保持梯度的幅值不变。实际上，这种变换并非线性，且归一化后的数值也不一定满足高斯分布或相互独立的条件，但我们依然认为批量归一化有助于改善梯度传播的表现。批量归一化对梯度传播的具体影响仍是未来需要进一步研究的课题。

3.4 批量归一化可对模型起到正则化作用

在采用批量归一化进行训练时，单个训练样本会与小批量中的其他样本一同被处理，且经过训练的网络不会再为某个特定的训练样本输出确定的数值。在我们的实验中，我们发现这种效应有助于提升网络的泛化能力。虽然 Dropout 机制（Srivastava等人，2014年）通常用于降低过拟合问题，但在批归一化网络中，我们发现既可以不使用该机制，也可以适当减弱其强度。

4 实验

4.1 激活值随时间的变化

为验证内部协变量偏移对训练过程的影响，以及批量归一化在应对这一问题上的作用，我们选择了在 MNIST 数据集（LeCun等人，1998a）上预测数字类别这一任务作为研究对象。我们构建了一个极为简单的网络，该网络以 28x28 像素的二值图像作为输入，

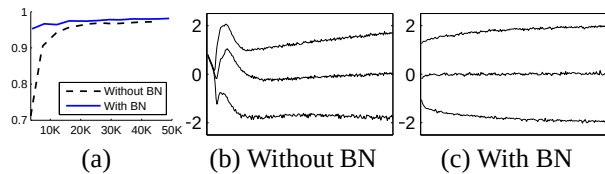


Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as $\{15, 50, 85\}$ th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*

3 fully-connected hidden layers with 100 activations each. Each hidden layer computes $y = g(Wu+b)$ with sigmoid nonlinearity, and the weights W initialized to small random Gaussian values. The last hidden layer is followed by a fully-connected layer with 10 activations (one per class) and cross-entropy loss. We trained the network for 50000 steps, with 60 examples per mini-batch. We added Batch Normalization to each hidden layer of the network, as in Sec. 3.1. We were interested in the comparison between the baseline and batch-normalized networks, rather than achieving the state of the art performance on MNIST (which the described architecture does not).

Figure 1(a) shows the fraction of correct predictions by the two networks on held-out test data, as training progresses. The batch-normalized network enjoys the higher test accuracy. To investigate why, we studied inputs to the sigmoid, in the original network N and batch-normalized network N_{BN}^{tr} (Alg. 2) over the course of training. In Fig. 1(b,c) we show, for one typical activation from the last hidden layer of each network, how its distribution evolves. The distributions in the original network change significantly over time, both in their mean and the variance, which complicates the training of the subsequent layers. In contrast, the distributions in the batch-normalized network are much more stable as training progresses, which aids the training.

4.2 ImageNet classification

We applied Batch Normalization to a new variant of the Inception network (Szegedy et al., 2014), trained on the ImageNet classification task (Russakovsky et al., 2014). The network has a large number of convolutional and pooling layers, with a softmax layer to predict the image class, out of 1000 possibilities. Convolutional layers use ReLU as the nonlinearity. The main difference to the network described in (Szegedy et al., 2014) is that the 5×5 convolutional layers are replaced by two consecutive layers of 3×3 convolutions with up to 128 filters. The network contains $13.6 \cdot 10^6$ parameters, and, other than the top softmax layer, has no fully-connected layers. More

details are given in the Appendix. We refer to this model as *Inception* in the rest of the text. The model was trained using a version of Stochastic Gradient Descent with momentum (Sutskever et al., 2013), using the mini-batch size of 32. The training was performed using a large-scale, distributed architecture (similar to (Dean et al., 2012)). All networks are evaluated as training progresses by computing the validation accuracy @1, i.e. the probability of predicting the correct label out of 1000 possibilities, on a held-out set, using a single crop per image.

In our experiments, we evaluated several modifications of Inception with Batch Normalization. In all cases, Batch Normalization was applied to the input of each nonlinearity, in a convolutional way, as described in section 3.2, while keeping the rest of the architecture constant.

4.2.1 Accelerating BN Networks

Simply adding Batch Normalization to a network does not take full advantage of our method. To do so, we further changed the network and its training parameters, as follows:

Increase learning rate. In a batch-normalized model, we have been able to achieve a training speedup from higher learning rates, with no ill side effects (Sec. 3.3).

Remove Dropout. As described in Sec. 3.4, Batch Normalization fulfills some of the same goals as Dropout. Removing Dropout from Modified BN-Inception speeds up training, without increasing overfitting.

Reduce the L_2 weight regularization. While in Inception an L_2 loss on the model parameters controls overfitting, in Modified BN-Inception the weight of this loss is reduced by a factor of 5. We find that this improves the accuracy on the held-out validation data.

Accelerate the learning rate decay. In training Inception, learning rate was decayed exponentially. Because our network trains faster than Inception, we lower the learning rate 6 times faster.

Remove Local Response Normalization While Inception and other networks (Srivastava et al., 2014) benefit from it, we found that with Batch Normalization it is not necessary.

Shuffle training examples more thoroughly. We enabled within-shard shuffling of the training data, which prevents the same examples from always appearing in a mini-batch together. This led to about 1% improvements in the validation accuracy, which is consistent with the view of Batch Normalization as a regularizer (Sec. 3.4): the randomization inherent in our method should be most beneficial when it affects an example differently each time it is seen.

Reduce the photometric distortions. Because batch-normalized networks train faster and observe each training example fewer times, we let the trainer focus on more “real” images by distorting them less.

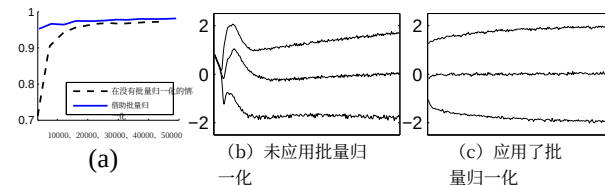


图1: (a) 展示了在应用与未应用批量归一化的情况下训练的MNIST网络对应的测试准确率随训练步数的变化情况。批量归一化有助于加快网络训练速度并提升准确率。(b、c) 显示了在训练过程中输入数据分布向典型Sigmoid函数形态的演变过程, 以 $\{15, 50, 85\}$ 分位数形式呈现。批量归一化能够使数据分布更加稳定, 同时减少内部协变量偏移。

网络包含3个全连接隐藏层, 每个层均拥有100个激活值。每个隐藏层都会通过Sigmoid非线性函数计算 $y = g(Wu+b)$, 其权重 W 则被初始化为较小的随机高斯数值。在最后一个隐藏层之后还设有一层全连接层, 该层拥有10个输出节点(对应10个数字类别), 并采用交叉熵损失函数。我们在训练过程中设置了50000步的迭代次数, 每小批量包含60个样本。正如3.1节所述, 我们在网络的每个隐藏层都加入了批量归一化处理。我们的研究重点在于对比基准网络与经过批量归一化处理的网络之间的性能差异, 而非追求在MNIST数据集上达到当前最优的识别性能——因为所描述的架构本身并无法实现这一目标。

图1(a)展示了随着训练的进行, 两种网络在保留的测试数据上正确预测的比例变化情况。可以发现, 批归一化网络的测试准确率更高。为了解释这一现象, 我们研究了在训练过程中, 原始网络N以及采用算法2构建的批归一化网络N中输入到Sigmoid函数的数值变化。图1(b)、(c)分别展示了对这两种网络最后一层隐藏层中某个典型激活值而言, 其分布随时间的变化趋势。原始网络中的这些分布无论是均值还是方差都会发生显著变化, 从而增加了后续层训练的难度; 相比之下, 批归一化网络中的分布在整个训练过程中都更为稳定, 这为训练提供了有利条件。

4.2 ImageNet图像分类任务

我们在基于ImageNet图像分类任务(Russakovsky等人, 2014年)训练的Inception网络新变体上应用了批量归一化技术(Szegedy等人, 2014年)。该网络包含大量卷积层与池化层, 最后通过一个softmax层从1000种可能的类别中预测出图像所属类别。这些卷积层采用ReLU函数作为非线性函数。与(Szegedy等人, 2014年)描述的原始网络相比, 其主要区别在于原有的卷积层被两层连续的 3×3 卷积操作层所替代, 每层最多包含128个滤波器。该模型拥有 $13.6 \cdot 10^6$ 个参数, 且除了最顶层的softmax层外, 不存在全连接层。更多细节请见附录。

相关详细内容收录在附录中。在后续论述中, 我们将该模型简称为Inceptionin。该模型的训练采用了带有动量的随机梯度下降法(Sutskever等人, 2013年), 所设置的小批量大小为32。训练过程借助大规模分布式架构来完成(类似于Dean等人, 2012年的架构)。随着训练的推进, 我们会通过对保留测试集上的每张图像进行单次裁剪采样, 计算验证准确率@1来评估所有模型的性能, 该准确率指的是在1000种可能类别中预测出正确类别的概率。

在我们的实验中, 我们对结合批量归一化的Inception网络进行了多种改进方案的测试。在所有情况下, 均按照3.2节所述的方式, 以卷积方式对每个非线性函数的输入施加批量归一化处理, 同时保持网络架构的其他部分不变。

4.2.1 加速批量归一化模型的训练速度

仅仅在模型中加入批量归一化并不足以充分发挥本方法的优势。为此, 我们进一步对模型及其训练参数进行了调整, 具体如下:

*提高学习率。*在经过批量归一化处理的模型中, 较高的学习率能够提升训练速度, 且不会带来任何负面效应(见第3.3节)。

*移除Dropout机制。*如第3.4节所述, 批量归一化已能实现与Dropout机制部分相同的目标。在改良后的BN-Inception模型中移除Dropout机制后, 不仅提升了训练速度, 还降低了过拟合的风险。

*降低 L_2 权重正则化的强度。*在Inception网络中, 是通过针对模型参数的 L_2 损失函数来控制过拟合现象的, 而在改良后的BN-Inception模型中, 该损失函数的权重被降低了5倍。实验表明, 这样做能够提升用于验证的独立数据集上的准确率。

*加快学习率衰减的速度。*在训练Inception网络时, 学习率是采用指数方式衰减的。由于我们的模型训练速度更快, 因此学习率的衰减速度也相应快了6倍。

虽然Inception及其他网络(Srivastava等人, 2014年)能从局部响应归一化处理中受益, 但我们在使用批量归一化的情况下发现并无此必要。

我们对训练样本进行了更彻底的打乱处理。我们启用了训练数据在各个分片内的随机排序功能, 从而避免相同的训练样本总是出现在同一个小批量中。这一措施使得验证准确率提升了约1%, 这一结果与将批量归一化视为正则化技术的观点相符(见第3.4节): 由于我们的方法具有随机性, 当每次处理同一个样本时都能使其呈现不同状态, 所带来的益处将会最为显著。

通过降低光度失真, 可以有效减少这类问题。由于经过批量归一化的网络训练速度更快, 且需要遍历每个训练样本的次数更少, 因此我们通过减少图像的失真程度, 让训练器能够专注于处理更“真实”的图像。

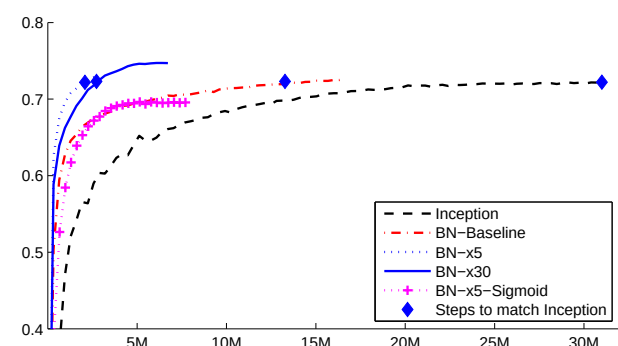


Figure 2: Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.

4.2.2 Single-Network Classification

We evaluated the following networks, all trained on the LSVRC2012 training data, and tested on the validation data:

Inception: the network described at the beginning of Section 4.2, trained with the initial learning rate of 0.0015.

BN-Baseline: Same as Inception with Batch Normalization before each nonlinearity.

BN-x5: Inception with Batch Normalization and the modifications in Sec. 4.2.1. The initial learning rate was increased by a factor of 5, to 0.0075. The same learning rate increase with original Inception caused the model parameters to reach machine infinity.

BN-x30: Like *BN-x5*, but with the initial learning rate 0.045 (30 times that of Inception).

BN-x5-Sigmoid: Like *BN-x5*, but with sigmoid nonlinearity $g(t) = \frac{1}{1+\exp(-x)}$ instead of ReLU. We also attempted to train the original Inception with sigmoid, but the model remained at the accuracy equivalent to chance.

In Figure 2, we show the validation accuracy of the networks, as a function of the number of training steps. Inception reached the accuracy of 72.2% after $31 \cdot 10^6$ training steps. The Figure 3 shows, for each network, the number of training steps required to reach the same 72.2% accuracy, as well as the maximum validation accuracy reached by the network and the number of steps to reach it.

By only using Batch Normalization (*BN-Baseline*), we match the accuracy of Inception in less than half the number of training steps. By applying the modifications in Sec. 4.2.1, we significantly increase the training speed of the network. *BN-x5* needs 14 times fewer steps than Inception to reach the 72.2% accuracy. Interestingly, increasing the learning rate further (*BN-x30*) causes the model to train somewhat *slower* initially, but allows it to reach a higher final accuracy. It reaches 74.8% after $6 \cdot 10^6$ steps, i.e. 5 times fewer steps than required by Inception to reach 72.2%.

We also verified that the reduction in internal covariate shift allows deep networks with Batch Normalization

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
<i>BN-Baseline</i>	$13.3 \cdot 10^6$	72.7%
<i>BN-x5</i>	$2.1 \cdot 10^6$	73.0%
<i>BN-x30</i>	$2.7 \cdot 10^6$	74.8%
<i>BN-x5-Sigmoid</i>		69.8%

Figure 3: For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.

to be trained when sigmoid is used as the nonlinearity, despite the well-known difficulty of training such networks. Indeed, *BN-x5-Sigmoid* achieves the accuracy of 69.8%. Without Batch Normalization, Inception with sigmoid never achieves better than 1/1000 accuracy.

4.2.3 Ensemble Classification

The current reported best results on the ImageNet Large Scale Visual Recognition Competition are reached by the Deep Image ensemble of traditional models (Wu et al., 2015) and the ensemble model of (He et al., 2015). The latter reports the top-5 error of 4.94%, as evaluated by the ILSVRC server. Here we report a top-5 validation error of 4.9%, and test error of 4.82% (according to the ILSVRC server). This improves upon the previous best result, and exceeds the estimated accuracy of human raters according to (Russakovsky et al., 2014).

For our ensemble, we used 6 networks. Each was based on *BN-x30*, modified via some of the following: increased initial weights in the convolutional layers; using Dropout (with the Dropout probability of 5% or 10%, vs. 40% for the original Inception); and using non-convolutional, per-activation Batch Normalization with last hidden layers of the model. Each network achieved its maximum accuracy after about $6 \cdot 10^6$ training steps. The ensemble prediction was based on the arithmetic average of class probabilities predicted by the constituent networks. The details of ensemble and multicrop inference are similar to (Szegedy et al., 2014).

We demonstrate in Fig. 4 that batch normalization allows us to set new state-of-the-art by a healthy margin on the ImageNet classification challenge benchmarks.

5 Conclusion

We have presented a novel mechanism for dramatically accelerating the training of deep networks. It is based on the premise that covariate shift, which is known to complicate the training of machine learning systems, also ap-

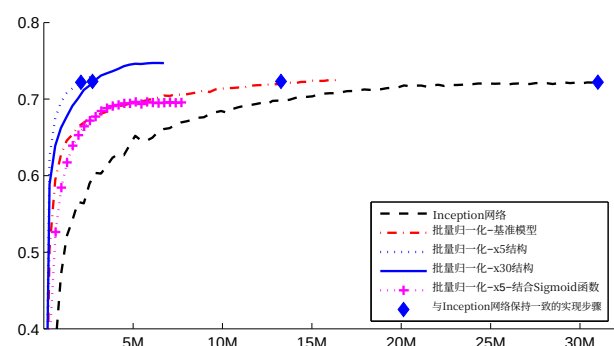


图2: Inception网络及其经过批量归一化处理的变体的单次训练集验证准确率与训练步数的关系。

4.2.2 单网络分类

我们评估了以下这些网络，它们均是在LSVRC2012训练数据集上训练的，并在验证数据集上进行了测试：

Inception网络即第4.2节开头所介绍的模型，其初始学习率为0.0015。

BN基线版本：与Inception网络结构相同，只是

在每个非线性函数之前加入了批量归一化处理。BN-x5版本是在Inception网络的基础上引入批量归一化处理，并采用4.2.1节中所述的改进方案。初始学习率被提升了5倍，达到了0.0075。与原始Inception网络相同的学习率提升幅度导致模型参数出现了数值无穷大的情况。

BN-x30版本与BN-x5版本类似，但其初始学习率为0.045，是原始Inception网络学习率的30倍。BN-x5-Sigmoid版本同样基于Inception网络，不同之处在于它使用了Sigmoid函数 $g(t) = \frac{1}{1 + \exp(-x)}$ 来替代ReLU函数。我们也曾尝试让原始Inception网络使用Sigmoid函数进行训练，但该模型的准确率仍仅相当于随机猜测的水平。

图2展示了这些模型在不同训练步数下的验证准确率变化情况。经过 $31 \cdot 10^6$ 步训练后，Inception网络的准确率达到了72.2%。图3则显示了每种模型为达到同样的72.2%准确率所需的训练步数、所能达到的最高验证准确率以及达到该最大准确率所需的步数。

仅使用BN基线版本时，我们就在不到Inception一半的训练步数下就实现了与其相同的准确率。通过应用4.2.1节中提出的改进方法，该网络的训练速度得到了显著提升——BN-x5达到72.2%准确率所需的训练步数仅为Inception的1/14。有趣的是，进一步提高学习率（即BN-x30版本）虽然会使得模型在初期训练速度稍慢，但却能使其最终达到的准确率更高，即在 $6 \cdot 10^6$ 步后就能达到74.8%的准确率，这一数值相当于Inception达到72.2%准确率所需步数的1/5。

我们还验证了，通过批量归一化处理来减少内部协变量偏移，确实能够提升深度神经网络的性能。

模型	达成72.2%目标的步骤	最大准确率
Inception网络	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid版本		69.8%

图3: 针对Inception网络及其经过批量归一化处理的变体，展示了达到Inception网络最高准确率（72.2%）所需的训练步数，以及这些网络实际能够达到的最高准确率。

的确如此，尽管利用Sigmoid函数作为非线性函数的深度网络训练起来极为困难，但批量归一化的存在使得这类网络仍可有效训练。具体而言，BN-x5-Sigmoid版本的准确率达到了69.8%，而未采用批量归一化的含Sigmoid函数的Inception网络，其准确率始终无法超越 1/1000这一数值。

4.2.3 集成模型分类

目前，在ImageNet大规模视觉识别竞赛中取得的最佳成绩是由传统模型的Deep Image集成模型（吴等人，2015年）以及何等人2015年提出的集成模型所创造的。后者经ILSVRC评测服务器评估后的前5名错误率为4.94%。而我们的研究结果中，验证集的前5名错误率为4.9%，测试集的错误率为4.82%（同样依据ILSVRC评测服务器的数据）。这一成绩不仅优于此前的最佳纪录，还超过了Russakovsky等人2014年预测的人工评估员的准确率水平。

在构建我们的集成模型时，我们共使用了6个模型。这些模型均基于BN-x30版本，并通过以下方式进行了改进：增加卷积层中的初始权重；采用Dropout机制（其概率分别为5%和10%，而原始的Inception网络则采用40%的概率）；以及在模型的最后几层隐藏层中应用非卷积式的、针对每个激活值的批量归一化处理。每个模型在经过大约 $6 \cdot 10^6$ 训练步数后都能达到最高的准确率。该集成模型的预测结果则是根据各个组成模型所输出的类别概率计算得出的算术平均值。关于集成模型与多图像推理的详细内容，可参考(Szegedy等人，2014年的研究)。

图4展示了批量归一化技术如何帮助我们在ImageNet图像分类任务的各项基准测试中，以较大的优势创造出当前最佳性能模型。

5 结论

我们提出了一种全新的机制，能够显著加速深度网络训练的速度。该机制的核心理念是：众所周知，会加剧机器学习系统训练复杂度的协变量偏移现象，同样也会影响子网络与各层的功能表现，而消除这种偏移对网络内部激活值的影响，有望为训练过程带来助力。

Model	Resolution	Crops	Models	Top-1 error	Top-5 error
GoogLeNet ensemble	224	144	7	-	6.67%
Deep Image low-res	256	-	1	-	7.96%
Deep Image high-res	512	-	1	24.88	7.42%
Deep Image ensemble	variable	-	-	-	5.98%
BN-Inception single crop	224	1	1	25.2%	7.82%
BN-Inception multicrop	224	144	1	21.99%	5.82%
BN-Inception ensemble	224	144	6	20.1%	4.9%*

Figure 4: *Batch-Normalized Inception comparison with previous state of the art on the provided validation set comprising 50000 images. *BN-Inception ensemble has reached 4.82% top-5 error on the 100000 images of the test set of the ImageNet as reported by the test server.*

plies to sub-networks and layers, and removing it from internal activations of the network may aid in training. Our proposed method draws its power from normalizing activations, and from incorporating this normalization in the network architecture itself. This ensures that the normalization is appropriately handled by any optimization method that is being used to train the network. To enable stochastic optimization methods commonly used in deep network training, we perform the normalization for each mini-batch, and backpropagate the gradients through the normalization parameters. Batch Normalization adds only two extra parameters per activation, and in doing so preserves the representation ability of the network. We presented an algorithm for constructing, training, and performing inference with batch-normalized networks. The resulting networks can be trained with saturating nonlinearities, are more tolerant to increased training rates, and often do not require Dropout for regularization.

Merely adding Batch Normalization to a state-of-the-art image classification model yields a substantial speedup in training. By further increasing the learning rates, removing Dropout, and applying other modifications afforded by Batch Normalization, we reach the previous state of the art with only a small fraction of training steps – and then beat the state of the art in single-network image classification. Furthermore, by combining multiple models trained with Batch Normalization, we perform better than the best known system on ImageNet, by a significant margin.

Interestingly, our method bears similarity to the standardization layer of (Gülçehre & Bengio, 2013), though the two methods stem from very different goals, and perform different tasks. The goal of Batch Normalization is to achieve a stable distribution of activation values throughout training, and in our experiments we apply it before the nonlinearity since that is where matching the first and second moments is more likely to result in a stable distribution. On the contrary, (Gülçehre & Bengio, 2013) apply the standardization layer to the *output* of the nonlinearity, which results in sparser activations. In our large-scale image classification experiments, we have not observed the nonlinearity *inputs* to be sparse, neither with nor without Batch Normalization. Other notable differ-

entiating characteristics of Batch Normalization include the learned scale and shift that allow the BN transform to represent identity (the standardization layer did not require this since it was followed by the learned linear transform that, conceptually, absorbs the necessary scale and shift), handling of convolutional layers, deterministic inference that does not depend on the mini-batch, and batch-normalizing each convolutional layer in the network.

In this work, we have not explored the full range of possibilities that Batch Normalization potentially enables. Our future work includes applications of our method to Recurrent Neural Networks (Pascanu et al., 2013), where the internal covariate shift and the vanishing or exploding gradients may be especially severe, and which would allow us to more thoroughly test the hypothesis that normalization improves gradient propagation (Sec. 3.3). We plan to investigate whether Batch Normalization can help with domain adaptation, in its traditional sense – i.e. whether the normalization performed by the network would allow it to more easily generalize to new data distributions, perhaps with just a recomputation of the population means and variances (Alg. 2). Finally, we believe that further theoretical analysis of the algorithm would allow still more improvements and applications.

References

Bengio, Yoshua and Glorot, Xavier. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of AISTATS 2010*, volume 9, pp. 249–256, May 2010.

Dean, Jeffrey, Corrado, Greg S., Monga, Rajat, Chen, Kai, Devin, Matthieu, Le, Quoc V., Mao, Mark Z., Ranzato, Marc’Aurelio, Senior, Andrew, Tucker, Paul, Yang, Ke, and Ng, Andrew Y. Large scale distributed deep networks. In *NIPS*, 2012.

Desjardins, Guillaume and Kavukcuoglu, Koray. Natural neural networks. (unpublished).

Duchi, John, Hazan, Elad, and Singer, Yoram. Adaptive subgradient methods for online learning and stochastic

模型	分辨率	裁剪区域	模型们	Top-1误差	前5名错误率
GoogLeNet集成模型	224	144	7	-	6.67%
Deep Image低分辨率版本	256	-	1	-	7.96%
Deep Image高分辨率版本	512	-	1	24.88	7.42%
Deep Image集成模型	变量	-	-	-	5.98%
BN-Inception单样本版本	224	1	1	25.2%	7.82%
BN-Inception多样本版本	224	144	1	21.99%	5.82%
BN-Inception集成模型	224	144	6	20.1%	4.9%*

图4：在包含50000张图像的验证集上，经过批量归一化的Inception模型与以往最先进模型的性能对比。据测试服务器显示，该BN-Inception集成模型在ImageNet测试集的100000张图像上的前5名错误率为4.82%。

我们所提出的方法之所以能有效提升训练效率，得益于对激活值进行归一化处理，并将这种归一化机制直接融入网络架构之中。这样一来，无论采用何种网络训练优化方法，都能妥善处理归一化操作。为了适配深度网络训练中常用的随机优化方法，我们会针对每个小批量数据执行归一化处理，再通过归一化参数反向传播梯度。批量归一化仅为每个激活值增加两个额外的参数，同时还能保留网络原有的表示能力。我们还提出了用于构建、训练以及运用批归一化网络进行推理的算法。通过该算法训练得到的网络不仅可以使使用饱和非线性函数，对更高的训练速率也有更强的承受能力，通常还无需借助Dropout机制来实现正则化处理。

仅在最先进的图像分类模型中加入批量归一化处理，就能显著提升训练速度。进一步调整学习率、取消Dropout机制，并运用批量归一化带来的其他优化手段后，我们仅需极少的训练步数即可达到先前的最优秀水平，甚至在单网络图像分类领域超越了现有最佳成果。此外，通过整合多个经过批量归一化训练的模型，我们的性能更是大幅超越了目前ImageNet上已知的最优系统。

有趣的是，我们的方法与(Gülçehre & Bengio, 2013)提出的标准化层存在相似之处，尽管这两种方法的出发点截然不同，所承担的任务也各有差异。批量归一化的目标是在训练过程中让激活值的分布保持稳定，而在我们的实验中，我们会在非线性函数之前应用它，因为正是在这一环节，通过匹配二阶矩更有可能实现稳定的激活值分布。相反，(Gülçehre & Bengio, 2013)则将标准化层应用于非线性函数的输出，这样一来产生的激活值就会更为稀疏。在我们进行的大规模图像分类实验中，无论是否使用批量归一化，我们都未观察到非线性函数的输入会出现稀疏现象。除此之外，批量归一化还有其他一些值得注意的特点。

批量归一化的显著特点包括：能够通过学习得到的缩放参数和偏移量使得批归一化变换具有恒等变换的效果（而标准化层则无需如此，因为它之后紧接着的就是能够从概念上吸收所需缩放参数和偏移量的学习型线性变换）；能够适配卷积层；具备不依赖于小批量的确定性推理能力；以及能够对网络中的每一层卷积层都进行批量归一化处理。

在本研究中，我们尚未探索批量归一化所能实现的全部潜力。未来的工作包括将该方法应用于循环神经网络（Pascanu等人，2013年），因为在这类网络中内部协变量偏移以及梯度消失或梯度爆炸现象可能更为严重，这有助于我们更全面地验证归一化处理能够改善梯度传播这一假设（见第3.3节）。我们还计划研究批量归一化是否有助于传统意义上的领域自适应——也就是探讨网络所进行的归一化处理是否能让其更容易泛化到新的数据分布，或许只需重新计算总体均值和方差即可实现（算法2）。最后，我们认为对该算法进行更深入的理论分析，还将带来更多的改进与应用可能。

参考文献

Yoshua Bengio与Xavier Glorot合著的《深度前馈神经网络训练难度研究》，收录于2010年AISTATS会议论文集第9卷，页码249–256，出版时间为2010年5月。

Jeffrey Dean, Greg S. Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Quoc V. Le, Mark Z. Mao, Marc’Aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang以及Andrew Y. Ng共同撰写的《大规模分布式深度神经网络》，发表于2012年的NIPS会议论文集中。

Guillaume Desjardins与Koray Kavukcuoglu所著的《自然神经网络》（未公开发表）。

Duchi, John, Hazan, Elad以及Singer, Yoram。用于在线学习与随机过程的自适应次梯度方法

optimization. *J. Mach. Learn. Res.*, 12:2121–2159, July 2011. ISSN 1532-4435.

Gülçehre, Çağlar and Bengio, Yoshua. Knowledge matters: Importance of prior information for optimization. *CoRR*, abs/1301.4083, 2013.

He, K., Zhang, X., Ren, S., and Sun, J. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *ArXiv e-prints*, February 2015.

Hyvärinen, A. and Oja, E. Independent component analysis: Algorithms and applications. *Neural Netw.*, 13(4-5):411–430, May 2000.

Jiang, Jing. A literature survey on domain adaptation of statistical classifiers, 2008.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998a.

LeCun, Y., Bottou, L., Orr, G., and Muller, K. Efficient backprop. In Orr, G. and K., Muller (eds.), *Neural Networks: Tricks of the trade*. Springer, 1998b.

Lyu, S and Simoncelli, E P. Nonlinear image representation using divisive normalization. In *Proc. Computer Vision and Pattern Recognition*, pp. 1–8. IEEE Computer Society, Jun 23-28 2008. doi: 10.1109/CVPR.2008.4587821.

Nair, Vinod and Hinton, Geoffrey E. Rectified linear units improve restricted boltzmann machines. In *ICML*, pp. 807–814. Omnipress, 2010.

Pascanu, Razvan, Mikolov, Tomas, and Bengio, Yoshua. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013*, pp. 1310–1318, 2013.

Povey, Daniel, Zhang, Xiaohui, and Khudanpur, Sanjeev. Parallel training of deep neural networks with natural gradient and parameter averaging. *CoRR*, abs/1410.7455, 2014.

Raiko, Tapani, Valpola, Harri, and LeCun, Yann. Deep learning made easier by linear transformations in perceptrons. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 924–932, 2012.

Russakovsky, Olga, Deng, Jia, Su, Hao, Krause, Jonathan, Satheesh, Sanjeev, Ma, Sean, Huang, Zhiheng, Karpathy, Andrej, Khosla, Aditya, Bernstein, Michael, Berg, Alexander C., and Fei-Fei, Li. ImageNet Large Scale Visual Recognition Challenge, 2014.

Saxe, Andrew M., McClelland, James L., and Ganguli, Surya. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *CoRR*, abs/1312.6120, 2013.

Shimodaira, Hidetoshi. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244, October 2000.

Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: A simple way to prevent neural networks from overfitting. *J. Mach. Learn. Res.*, 15(1):1929–1958, January 2014.

Sutskever, Ilya, Martens, James, Dahl, George E., and Hinton, Geoffrey E. On the importance of initialization and momentum in deep learning. In *ICML (3)*, volume 28 of *JMLR Proceedings*, pp. 1139–1147. JMLR.org, 2013.

Szegedy, Christian, Liu, Wei, Jia, Yangqing, Sermanet, Pierre, Reed, Scott, Anguelov, Dragomir, Erhan, Dumitru, Vanhoucke, Vincent, and Rabinovich, Andrew. Going deeper with convolutions. *CoRR*, abs/1409.4842, 2014.

Wiesler, Simon and Ney, Hermann. A convergence analysis of log-linear training. In Shawe-Taylor, J., Zemel, R.S., Bartlett, P., Pereira, F.C.N., and Weinberger, K.Q. (eds.), *Advances in Neural Information Processing Systems 24*, pp. 657–665, Granada, Spain, December 2011.

Wiesler, Simon, Richard, Alexander, Schlüter, Ralf, and Ney, Hermann. Mean-normalized stochastic gradient for large-scale deep learning. In *IEEE International Conference on Acoustics, Speech, and Signal Processing*, pp. 180–184, Florence, Italy, May 2014.

Wu, Ren, Yan, Shengen, Shan, Yi, Dang, Qingqing, and Sun, Gang. Deep image: Scaling up image recognition, 2015.

Appendix

Variant of the Inception Model Used

Figure 5 documents the changes that were performed compared to the architecture with respect to the GoogleNet architecture. For the interpretation of this table, please consult (Szegedy et al., 2014). The notable architecture changes compared to the GoogLeNet model include:

- The 5×5 convolutional layers are replaced by two consecutive 3×3 convolutional layers. This increases the maximum depth of the network by 9

该文章详细探讨了自适应次梯度方法在在线学习及随机过程优化领域的应用。

Gülçehre, Çağlar与Yoshua Bengio共同撰写的《知识问题：优化过程中先验信息的重要性》一文发表于计算机科学检索库，论文编号为abs/1301.4083，发表时间为2013年。

何凯、张晓辉、任森和孙杰在《深入研究整流器：在ImageNet图像分类任务中超越人类水平的表现》一文中指出，基于整流器的架构有助于提升模型性能。该ArXiv预印本发布于2015年2月。

A. Hyvärinen与E. Oja合著的《独立成分分析：算法与应用》一文刊登在《神经网络》期刊2000年5月的第13卷第4-5期上，页码范围为411–430。

蒋静于2008年发表了关于统计分类器领域自适应技术的文献综述。

杨立昆、博图、Yoshua Bengio以及帕夫纳在《基于梯度的学习方法在文档识别中的应用》一文中进行了相关研究，该文章发表于IEEE会议录1998年11月a版，第86卷第11期，页码范围为2278–2324。

扬·勒昆、博图、戈尔与穆勒共同提出了高效的反向传播算法。该文收录于戈尔与穆勒主编的《神经网络：行业实用技巧》一书，由施普林格出版社于1998年出版。吕晓峰与西蒙切利则探讨了利用分割归一化处理实现非线性图像表示的方法，相关成果发表在2008年6月23日至28日由IEEE计算机协会举办的《计算机视觉与模式识别会议论文集》上，页码为1–8，论文编号为doi: 10.1109/CVPR.2008.4587821。维诺德·奈尔与杰弗里·E·辛顿指出修正线性单元能够提升受限玻尔兹曼机的性能，相关内容刊载于ICML会议论文集中，页码为807–814，出版时间为2010年，由奥姆尼普雷斯出版社发行。拉兹万·帕斯卡努、托马斯·米科洛夫与Yoshua Bengio研究了训练循环神经网络的难度问题，论文发表于2013年6月16日至21日在美国乔治亚州亚特兰大市举行的第30届国际机器学习大会（即ICML 2013会议）上，页码为1310–1318。丹尼尔·波维、张晓辉与桑吉夫·库丹普尔提出了结合自然梯度与参数平均法对深度神经网络进行并行训练的方法，相关研究发表在计算机科学检索库上，论文编号为abs/1410.7455，时间为2014年。塔帕尼·莱科、哈里·瓦尔波拉与扬·勒昆认为，感知器中的线性变换能够简化深度学习的过程，该观点发表于2012年的人工智能与统计学国际会议（AISTATS）上，页码为924–932。奥尔加·鲁萨科夫斯基、贾佳、苏浩、乔纳森·克劳斯、桑吉夫·萨蒂什、肖恩·马、郑志恒、安德烈·卡尔帕西、阿迪蒂亚·科斯拉、迈克尔·伯恩斯坦、亚历山大·C·伯格以及李飞飞共同介绍了2014年的ImageNet大规模视觉识别挑战赛。

安德鲁·M·萨克斯、詹姆斯·L·麦克莱兰德与苏里亚·甘古利找到了深度线性神经网络中学习过程非线性动态问题的精确解，相关研究发表在计算机科学检索库上，论文编号为abs/1312.6120，时间为2013年。

秀敏俊。通过给对数似然函数加权来提升在协变量偏移条件下的预测推断性能。《统计规划与推断杂志》，第90卷第2期，页码227–244，2000年10月。

尼蒂什·斯里瓦斯塔瓦、杰弗里·辛顿、亚历克斯·克里兹夫斯基、伊利亚·苏茨克维尔以及鲁слан·萨拉胡特丁诺夫。Dropout机制：一种防止神经网络过拟合的简单方法。《机器学习研究杂志》，第15卷第1期，页码1929–1958，2014年1月。

伊利亚·苏茨克维尔、詹姆斯·马滕斯、乔治·E·达尔以及杰弗里·E·辛顿。关于深度学习中初始化参数与动量法的重要性。载于《国际机器学习大会》（ICML 3）相关内容，《机器学习研究杂志》会议录第28卷，页码1139–1147。JMLR.org，2013年。

克里斯蒂安·塞格迪、魏刘、杨青佳、皮埃尔·塞尔马内、斯科特·里德、德拉戈米尔·安格洛夫、杜米特鲁·埃尔汉、文森特·范霍克以及安德鲁·拉比诺维奇。利用卷积操作进一步深化模型深度。计算机科学检索库，文章编号abs/1409.4842，2014年。西蒙·维斯勒与赫尔曼·奈。对数线性训练的收敛速度分析。载于J·肖·泰勒、R·S·泽梅尔、P·巴特利特、F·C·N·佩雷拉及K·Q·温伯格主编的《第24届神经信息处理系统进展》一书，页码657–665，西班牙格拉纳达，2011年12月。西蒙·维斯勒、亚历山大·理查德、拉尔夫·施卢特以及赫尔曼·奈。用于大规模深度学习的均值归一化随机梯度方法。载于2014年5月在意大利佛罗伦萨召开的IEEE国际声学、语音与信号处理会议，页码180–184。任武、盛根燕、易珊、邓庆庆及孙刚。Deep Image集成模型：提升图像识别能力，2015年。

附录

所采用的Inception模型变体

图5展示了相较于原架构，在采用GoogleNet架构后所进行的各项修改。关于该表格的解读，可参考 (Szegedy等人, 2014)的研究。与GoogLeNet模型相比，显著的架构变化包括：

- 原有的 5×5 卷积层被替换为两个连续的 3×3 卷积层，这使得网络的最大深度增加了9。

weight layers. Also it increases the number of parameters by 25% and the computational cost is increased by about 30%.

- The number 28×28 inception modules is increased from 2 to 3.
- Inside the modules, sometimes average, sometimes maximum-pooling is employed. This is indicated in the entries corresponding to the pooling layers of the table.
- There are no across the board pooling layers between any two Inception modules, but stride-2 convolution/pooling layers are employed before the filter concatenation in the modules 3c, 4e.

Our model employed separable convolution with depth multiplier 8 on the first convolutional layer. This reduces the computational cost while increasing the memory consumption at training time.

此外，这还会使权重层的数量增加25%，同时计算成本也会上升约30%。

- Inception模块的数量从2增加到了3。
- 在这些模块内部，有时会采用平均池化操作，有时则采用最大池化操作；表格中对应于池化层的条目便说明了这一点。
- 任意两个Inception模块之间并不存在通用的池化层，但在模块3c和4e中，在对滤波器进行拼接之前，会使用步长为2的可分离卷积/池化层。

我们的模型在第一个卷积层上采用了深度乘数为8的可分离卷积，这种方式虽然能在降低计算成本的同时提升训练时的内存消耗。

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	double #3×3 reduce	double #3×3	Pool +proj
convolution*	7×7/2	112×112×64	1						
max pool	3×3/2	56×56×64	0						
convolution	3×3/1	56×56×192	1		64	192			
max pool	3×3/2	28×28×192	0						
inception (3a)		28×28×256	3	64	64	64	64	96	avg + 32
inception (3b)		28×28×320	3	64	64	96	64	96	avg + 64
inception (3c)	stride 2	28×28×576	3	0	128	160	64	96	max + pass through
inception (4a)		14×14×576	3	224	64	96	96	128	avg + 128
inception (4b)		14×14×576	3	192	96	128	96	128	avg + 128
inception (4c)		14×14×576	3	160	128	160	128	160	avg + 128
inception (4d)		14×14×576	3	96	128	192	160	192	avg + 128
inception (4e)	stride 2	14×14×1024	3	0	128	192	192	256	max + pass through
inception (5a)		7×7×1024	3	352	192	320	160	224	avg + 128
inception (5b)		7×7×1024	3	352	192	320	192	224	max + 128
avg pool	7×7/1	1×1×1024	0						

Figure 5: Inception architecture

type	块大小/ 步长	输出结果 size	深度	#1×1	#3×3 降低	#3×3	两倍于 #3×3 减少	加倍 #3×3	执行 +proj池化 操作
卷积运算	7×7/2	112×112×64	1						
最大值池化	3×3/2	56×56×64	0						
卷积运算	3×3/1	56×56×192	1		64	192			
最大值池化操作	3×3/2	28×28×192	0						
Inception模块3a版		28×28×256	3	64	64	64	64	96	平均值 + 32
Inception模块3b版		28×28×320	3	64	64	96	64	96	平均值 + 64
Inception模块3c版	步长为2	28×28×576	3	0	128	160	64	96	取 + 的最大值后直接传递
inception 4a		14×14×576	3	224	64	96	96	128	+ 128的平均值
inception 4b		14×14×576	3	192	96	128	96	128	+ 128的平均值
inception 4c		14×14×576	3	160	128	160	128	160	平均值 + 128
inception 4d		14×14×576	3	96	128	192	160	192	平均值 + 128
inception 4e	步长为2	14×14×1024	3	0	128	192	192	256	最大值 +, 直接传递
Inception网络 (5a)		7×7×1024	3	352	192	320	160	224	平均值 + 128
Inception网络 (5b) (参数版本)		7×7×1024	3	352	192	320	192	224	最大值 + 128
平均值池化操作	7×7/1	1×1×1024	0						

图5: Inception网络架构